云原生混部 Koordinator

– A QoS based scheduling system for hybrid orchestration workloads on Kubernetes

佑祎 吕风

阿里云容器服务团队

Koordinator 社区

2022/06/15

Contents

- 混部技术的介绍和发展
- Koordinator简介
- Quick Start
- 关键技术介绍
- 后续演进计划
- 社区建设

01 混部技术的介绍和发展

什么是混部

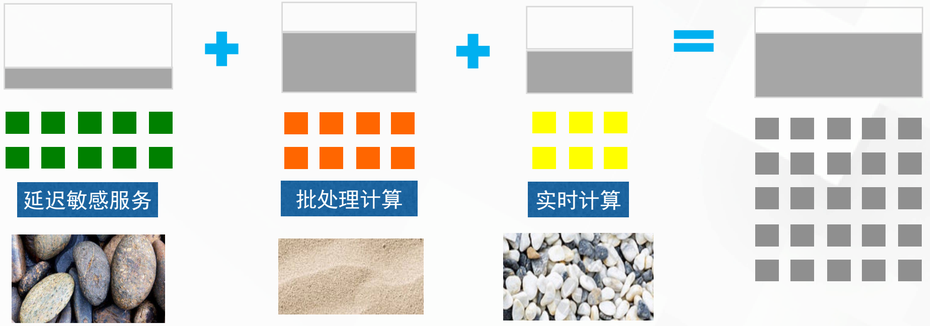
混部定义：

1，基于节点粒度的定义： **多个容器部署在一个节点上**，其中：

多个容器： 包括多个在线容器，多个离线容器，多个在离线容器三种形态，不仅仅是在离线容器；

一个节点：能够运行容器的最小单位，包括物理机、ECS等；

2，基于集群粒度的定义： 多种应用在一个集群内自动部署，通过预测分析应用特性，业务间错峰填谷；



混部的核心技术：

1，节点粒度：

a、容器隔离技术，包括依赖内核的袋鼠、RunC等；

b、单机调度技术：包括单机的负载感知，容器策略和阈值设置，容器优先级和伸缩控制、cpushare等；

c、中心调度技术：支持单机粒度的负载反馈、调度策略、调度性能等；

d、**资源的差异化SLO技术**： 包括差异化SLO 设定，优先级设定，基于差异化SLO 的单机和中心调度配合；

2，集群粒度：基于节点粒度之上实现；

a，中心调度技术：高性能调度、资源视图、多负载的调度协同、GPU拓扑感知等；

b，单机调度技术：任务的高频起停控制，单机多环境、硬件适配、cpu归一化等；

c，**k8s 生态框架优化和配套能力建设**，包括集群规模突破、稳定性改进、运维配套能力、界面和接口能力、业务对接等；

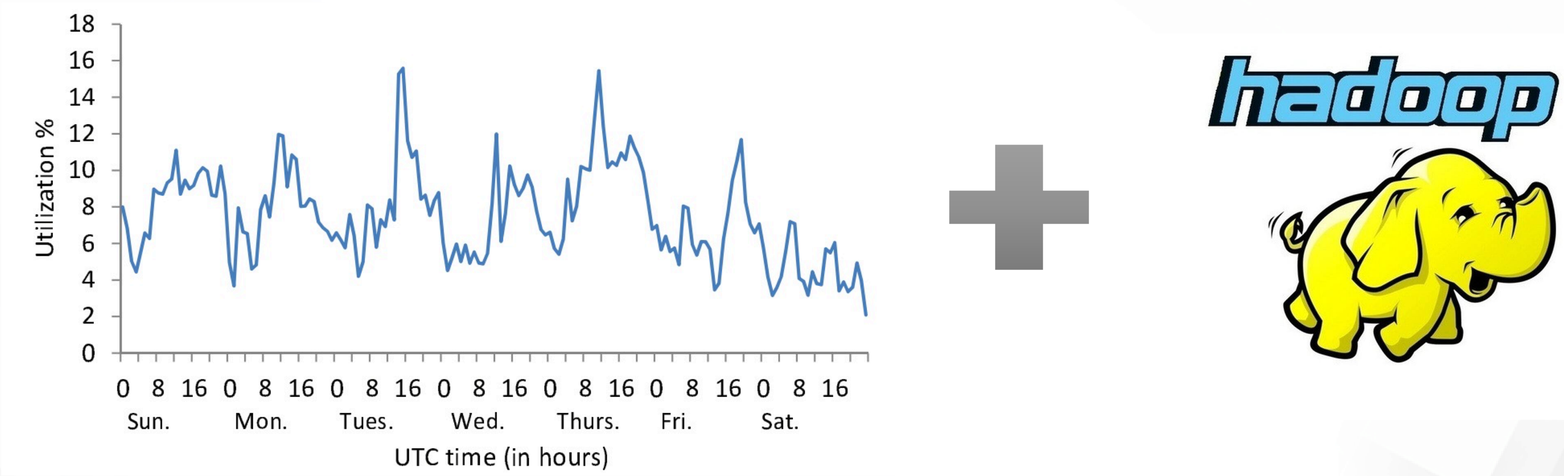
混部的目标和方案

混部的目标：

- 对数据中心**资源利用效率**的不懈追求
- 埃森哲报告显示 2011 年公有云数据中心的机器利用率平均不到 10%，意味着企业的资源成本极高
- 另一方面大数据技术的发展迅速，计算作业对资源的需求越来越大。

如何混部：

- **大数据云原生**上云已成必然趋势
  - 据Pepperdata 2021年12月调查报告。企业大数据平台开始向云原生技术迁移。超过77%的受访者预计到2021年底，其50%的大数据应用将迁移到 Kubernetes 平台。
- Google 公开数据显示其数据中心利用率 30%~40% @2011，50%～60% @2019
- 在线服务类型应用 + 批处理类型任务



混部需要解决的问题

集群管理

- 提升集群资源利用率，降低 IT 成本
- 简化对集群资源的管理，可以实现对各类应用的资源容量，分配量，使用量的清晰洞察

在线服务类型应用（延时敏感型，Latency Sensitive）：

- 容器混合部署时的互相干扰（noisy neighbor）
- 单机的资源竞应用对**资源的敏感程度**不同，在资源调度和单机服务质量上都需要精细化的管理策略
- 资源竞争引发应用响应时间出现抖动毛刺的现象，产生长尾问题（tail latency）

作业类型批处理任务（计算密集型）：

- 分级可靠的资源超卖，满足差异化的资源质量需求
- 及时**识别干扰源**，避免影响 LS 应用

Koordinator提供了以下机制：

- 面向混部场景的资源优先级和服务质量模型
- 稳定可靠的资源超卖机制
- 细粒度的容器资源编排和隔离机制
- 针对多种类型工作负载的调度能力增强
- 复杂类型工作负载的快速接入能力



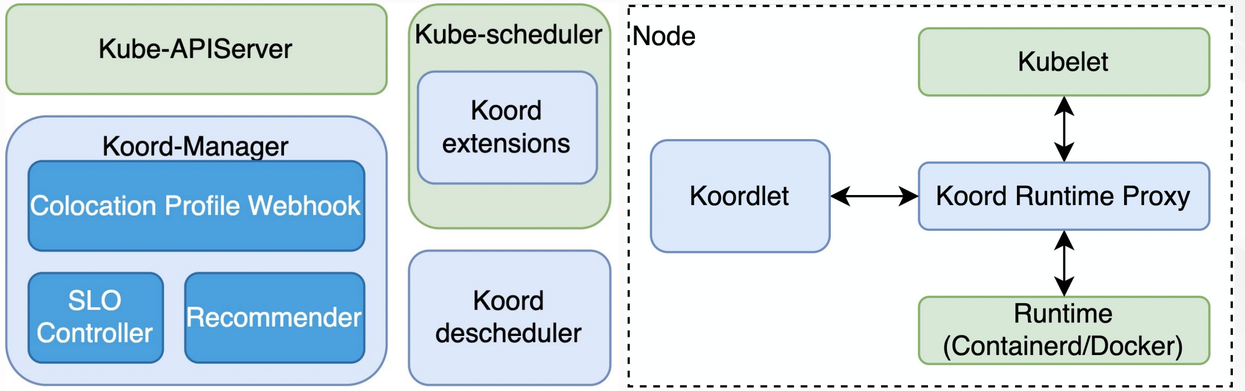
koordinator

02 Koordinator 简介

系统架构

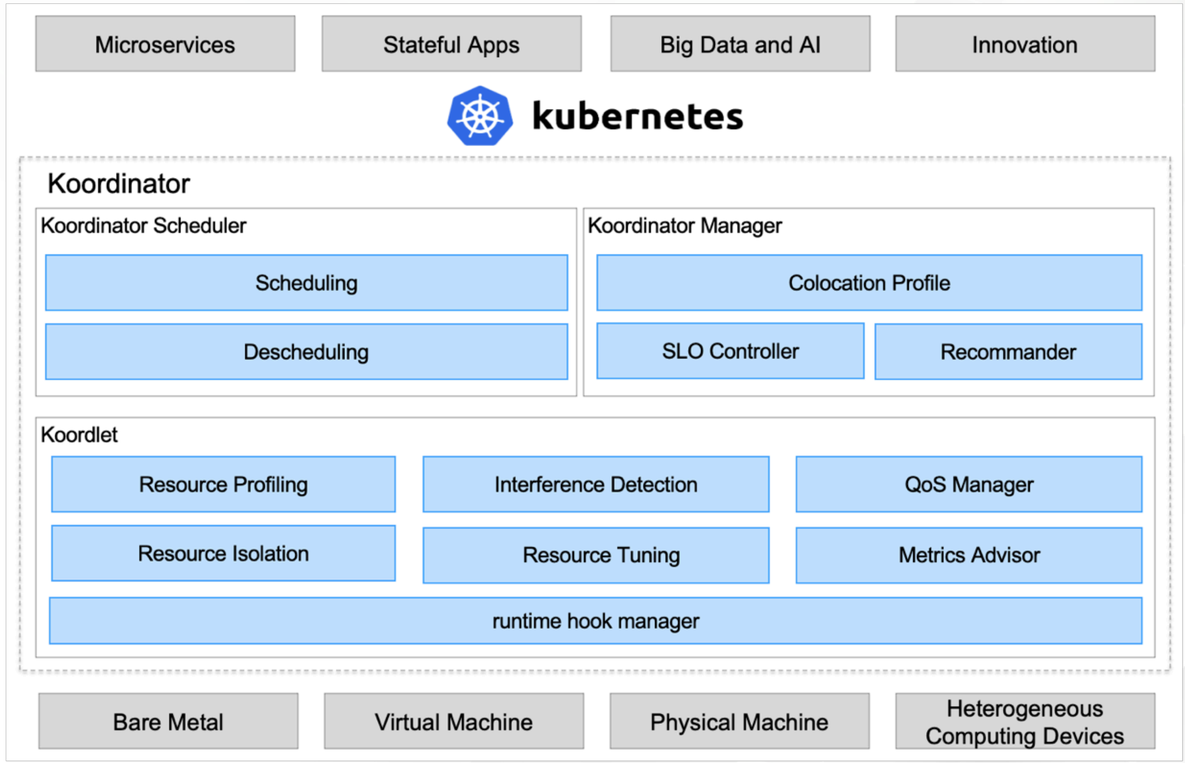
中心管控

- Koord-Manager
  - SLO-Controller
    - 提供**资源超卖**、混部 SLO 管理、精细化调度增强等核心管控能力
  - Recommender：围绕**资源画像**为应用提供相关的弹性能力
  - Colocation Profile Webhook
    - 简化 Koordinator 混部模型的使用
    - 为应用提供**一键接入**的能力，自动注入相关优先级、QoS 配置
- Koord extensions for Scheduler：面向**混部场景的调度能力**增强
- Koord descheduler：提供灵活可扩展的**重调度**机制



单机资源管理

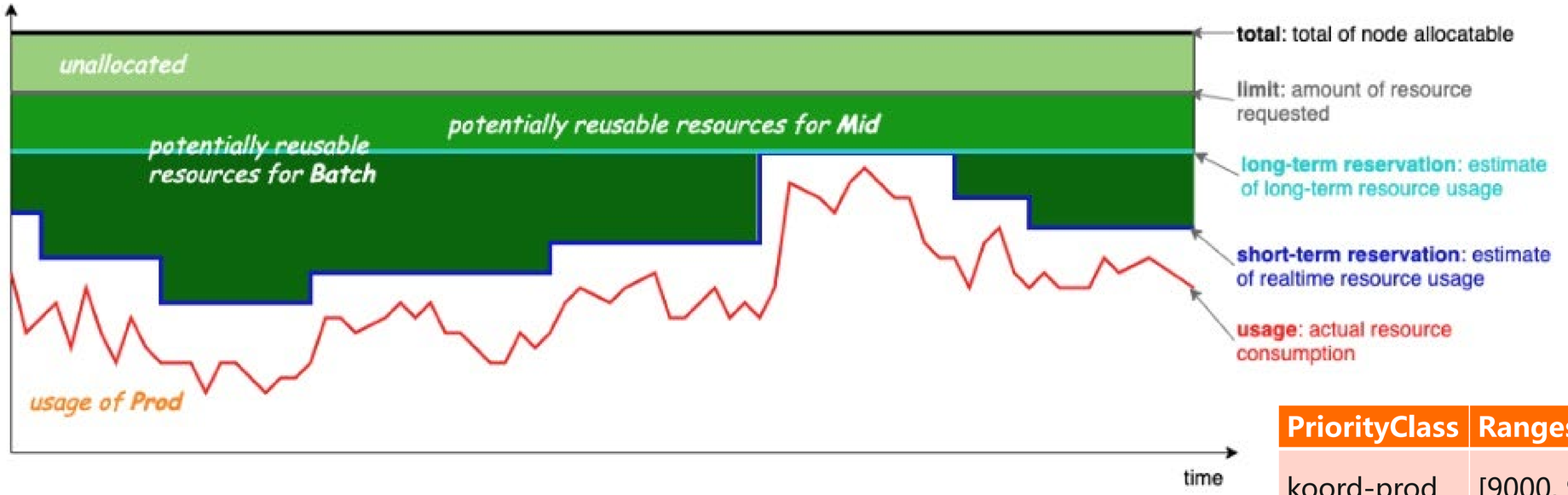
- Koord Runtime Proxy
  - 作为 Kubelet 和 Runtime 之间的**代理**，满足不同场景的资源管理需求
  - 提供插件化的**注册框架**，提供相关资源参数的注入机制
- Koordlet
  - 细粒度的**容器指标采集**，包括资源消耗、容器进程性能等
  - 面向不同 QoS 等级 Pod 的**干扰检测和调节**策略能力
  - 提供一系列的 **Runtime Proxy 插件**，支持精细化的 QoS 参数注入



Koordinator Priority

资源优先级

- 分为 **Product、Mid、Batch、Free** 四个等级，Pod 需要指定申请的资源优先级，scheduler 会基于总量和分配量做调度。
- 各优先级的资源总量会受高优先级资源的 request 和 usage 影响，例如**已申请但未使用**的 Product 资源会以 Batch 优先级再次分配。
- 资源**优先级策略的激进/保守**，决定了集群资源的超卖容量，与资源稳定性的高低成反相关
- Koordinator 将各优先级资源以标准的 **extend-resource** 形式更新在 Node 信息中



应用优先级

- 以 K8s **标准的 PriorityClass** 进行了扩展定义，代表 Pod 申请资源的优先级
- 同时提供了 Pod 级别的**子优先级**（sub-priority），用于调度器层面的精细化控制（排队，抢占等）
- Pod 不能跨优先级申请资源（例如不允许 Batch Pod 申请 Prod 资源）
- 多优先级资源超卖情况下，当单机资源紧张时，低优先级 Pod 会被压制或驱逐

PriorityClass	Ranges	Description
koord-prod	[9000, 9999]	典型的在在线应用，特征是对响应时间较为敏感
koord-mid	[7000, 7099]	流式计算，近线计算，AI 类型任务
koord-batch	[5000, 5999]	离线批处理任务，例如大数据作业、OLAP 类查询
koord-free	[3000, 3999]	低优先级的离线任务，常用于研发人员的测试场景

Koordinator QoS

服务质量（Quality of Service）

- 整体分为 System（系统级服务），Latency Sensitive（延迟敏感性的在线服务），Best Effort（资源消耗型的离线应用）三类
- 根据应用性能敏感程度的差异，Latency Sensitive 又细分为 LSE，LSR 和 LS
- 影响 Pod 在单机的**运行质量**，主要表现为使用的隔离参数不同，单机资源紧张时会优先满足高等级 QoS 的需求
- Koordinator 将 QoS 模型在 Pod Annotation 级别进行了扩展定义

Koordinator QoS	Description	Kubernetes QoS
System	系统级服务	--
LS (Latency Sensitive)	延迟敏感型应用，与其他 LS 类型应用共享 CPU 资源，弹性较好，对于突发的资源需求可以比较灵活的得到满足。	Guaranteed/Burstable
LSE (Latency Sensitive Exclusive)	敏感程度极高，要求独占 CPU 资源，不与其他任何 Pod 共享 CPU 核心。	Guaranteed
LSR (Latency Sensitive Reserved)	敏感程度介于 LS 和 LSE 之间，轻易不与其他任何 Pod 共享 CPU 核心。 与 LS 类型相比，LSR 对 CPU 的确定性要求更高；与 LSE 类型相比，在自身资源空闲的前提下，LSR 允许 BE 类型 Pod 临时使用。	Guaranteed
BE (Best Effort)	敏感程度较低的资源消耗型 Pod，在于以上类型应用混部时可以接受压制甚至驱逐，以避免对敏感型应用产生干扰。	BestEffort

Priority 与 QoS 的关系

关系描述

- Priority 与 QoS 是**正交**的两个维度，可以排列组合使用。
- 受模型定义和实际的需求情况影响，部分排列组合存在约束。

  （O表示合法的排列组合，X表示不合法的排列组合）

	Product	Mid	Batch	Free
LSE	O	X	X	X
LSR	O	X	X	X
LS	**O**	O	O	O
BE	X	O	**O**	O

场景举例

- 典型场景：
  - Prod + LS：典型的**在线应用**，通常对应用时延要求较高，对资源质量要求较高，也需要保证一定的资源弹性能力。
  - Batch + BE：用于混部场景中的**低优离线**，对资源质量有相当的忍耐度，例如批处理类型的 Spark/MR 任务，以及 AI 类型的训练任务
- 典型场景的增强：
  - Prod + LSR/LSE：比较**敏感的在线应用**，可以接受牺牲资源弹性而换取更好的确定性（如CPU绑核），对应用时延要求极高。
  - Mid/Free + BE：与“Batch + BE”相比主要区别是对资源质量要求的高低不同。
- 非典型的应用场景：
  - Mid/Batch/Free + LS：用于低优先级的在线服务、近线计算以及AI推理类等任务
    - 相较于大数据类型任务，它们无法接受过低的资源质量，对其他应用的干扰也相对较低
    - 而相较于典型的在线服务，它们又可以忍受相对较低的资源质量，例如接受一定程度的驱逐。

03 Quick Start

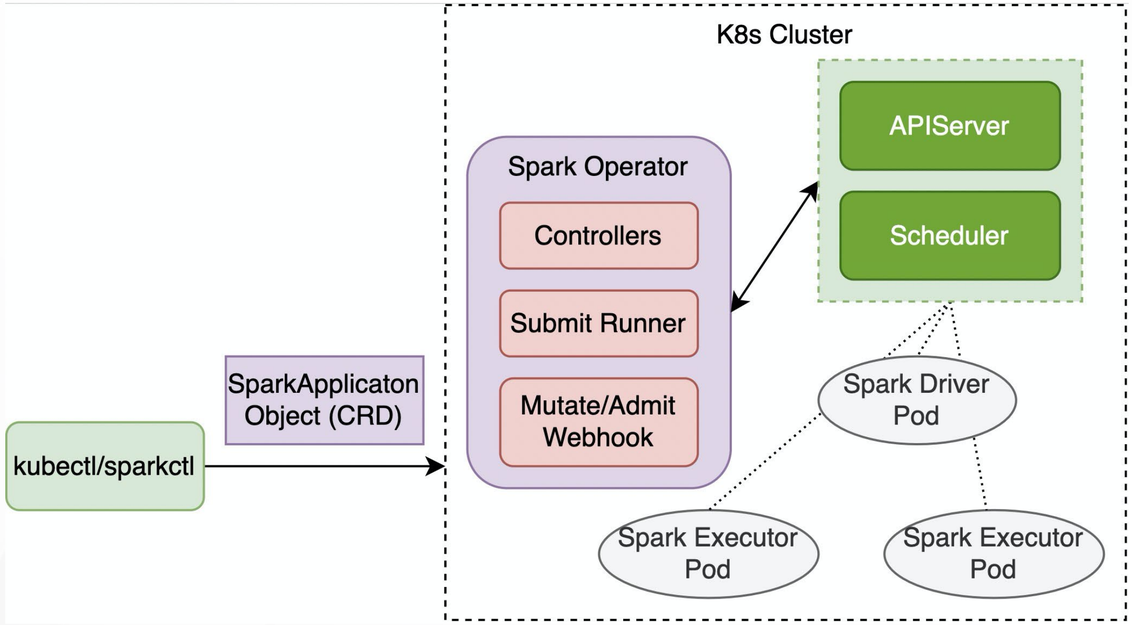
Spark on K8s 模式

通过 Spark Submit 提交

- 本地通过 Spark 客户端直接提交 Job 到 K8s 集群
- 简单快捷，常用于开发自测

通过 Spark Operator 提交

- 定义了 **SparkApplication CRD**，用于描述一个 Spark 作业
- 通过 kubectl 客户端将提交 SparkApplication CR 到 APIServer
- Spark Operator 负责作业生命周期以及 Driver Pod 的管理




```
# Spark Driver Pod example
apiVersion: v1
kind: Pod
metadata:
  labels:
    spark-role: driver
  ...
spec:
  containers:
  -  args:
      - driver
      - --properties-file
      - /opt/spark/conf/spark.properties
...
      resources:
        limits:
          cpu: 1
          memory: 3456Mi
        requests:
          cpu: 1
          memory: 3456Mi
  ...
```

Spark on K8s with Koordinator

ColocationProfile Webhook

- 向 Spark Pod **注入相关混部协议**
- QoS，Priority，extened-resource（batch-cpu/mem)

SLO Controller

- 混部超卖资源模型，更新 Node 的extended-resource
- 混部 QoS 策略管理

Koord Scheduler Extensions

- 调度能力增强，例如负载均衡等调度插件

Koordlet

- 单机资源质量保障
- 确保 Spark 混部过程后不影响在线应用性能表现

CPU 利用率提升

混部前

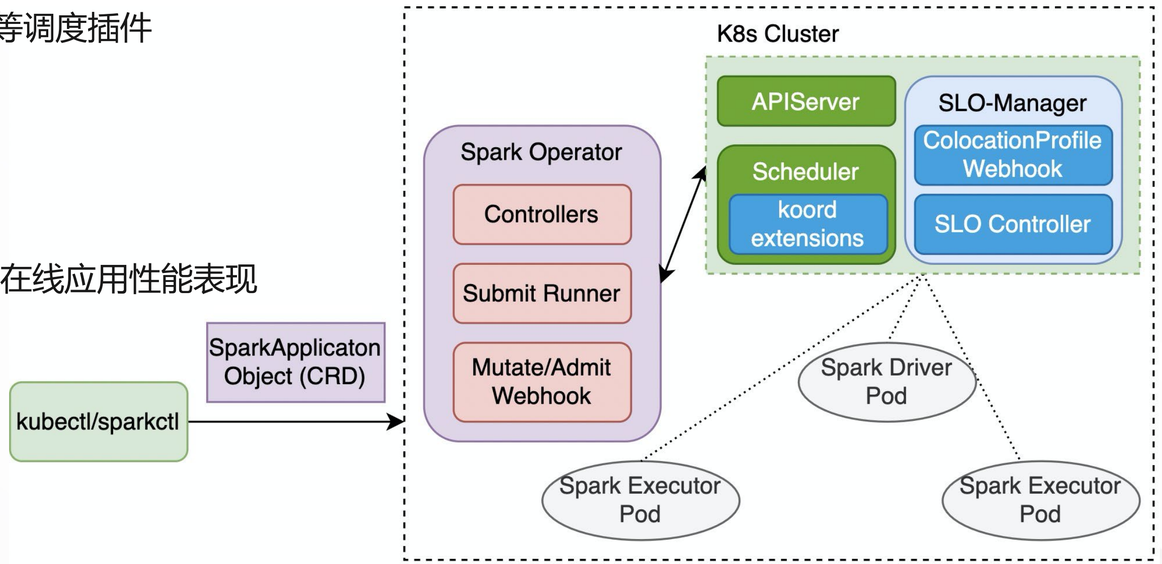
$ kubectl top node

NAME	CPU(cores)	CPU%
node-1	1190m	14.8%
node-2	1620m	20.25%

混部后

$ kubectl top node

NAME	CPU(cores)	CPU%
node-1	4077m	**52%**
node-2	3083m	**49%**

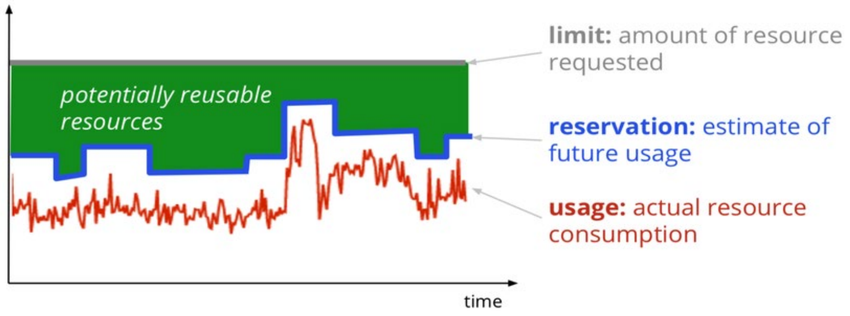


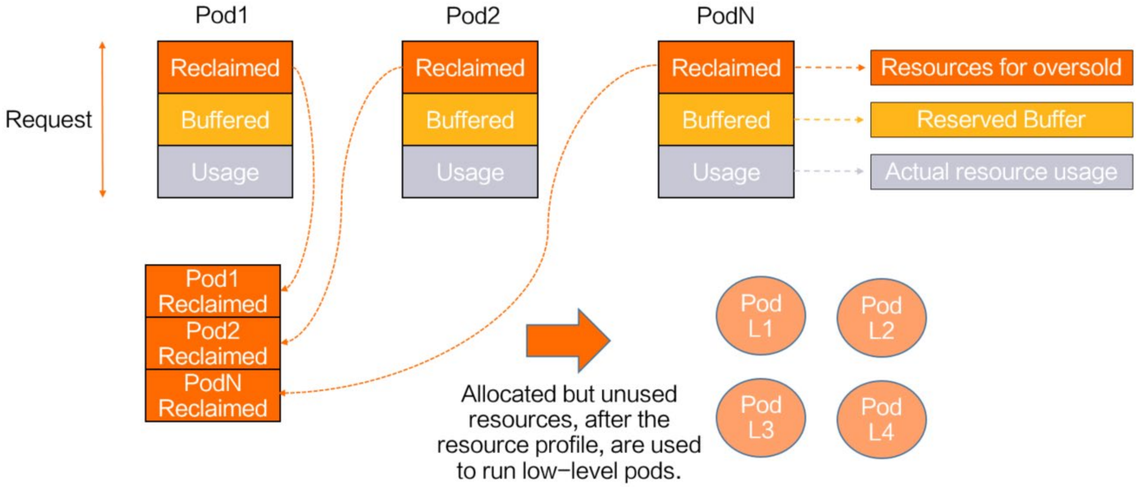

```
# Spark Driver Pod example
apiVersion: v1
kind: Pod
metadata:
  labels:
    koordinator.sh/qosClass: BE
...
spec:
  containers:
  -  args:
      - driver
...
resources:
        limits:
          koordinator.sh/batch-cpu: "1000"
          koordinator.sh/batch-memory: 3456Mi
        requests:
          koordinator.sh/batch-cpu: "1000"
          koordinator.sh/batch-memory: 3456Mi
  ...
```

04 关键技术介绍

资源超发 - Resource Overcommitment

Efficiency Resource reclamation



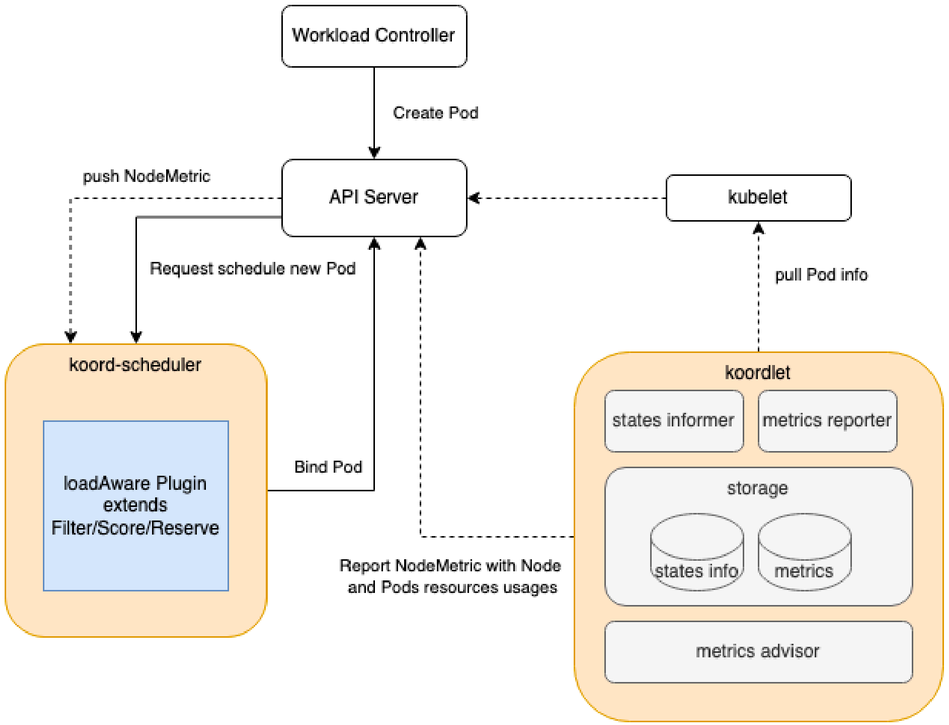


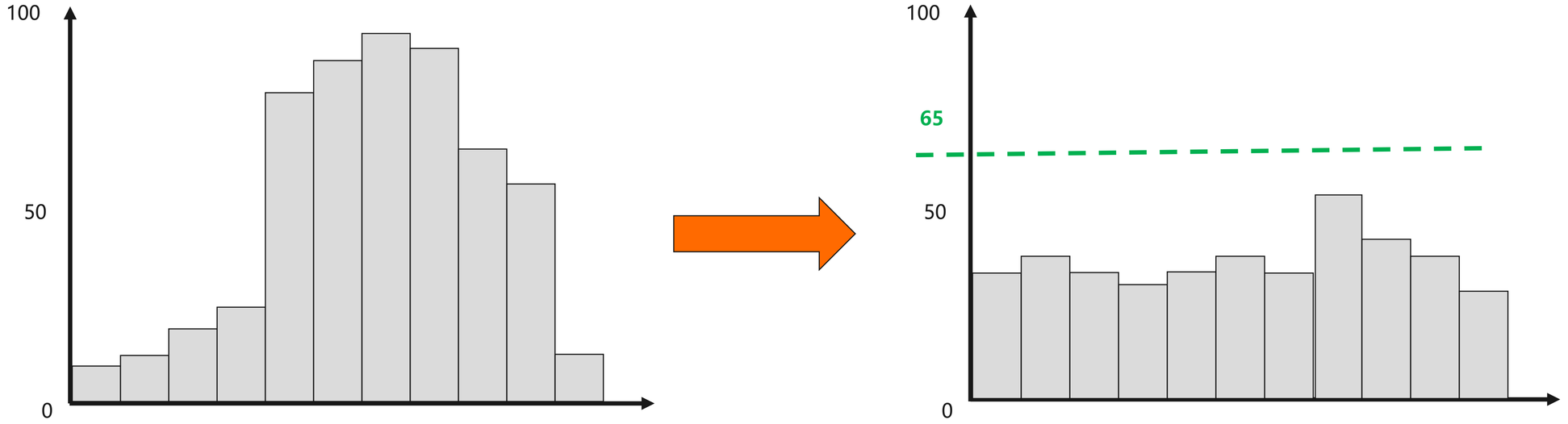

```
# node info
allocatable:
   koordinator.sh/bach-cpu: 50k # milli-core
   koordinator.sh/bach-memory: 50Gi
```

```
# pod info
annotations:
    koordinator.sh/resource-limit: {cpu:  “5k”  }
-   resources:
     requests
         koordinator.sh/bach-cpu: 5k # milli-core
         koordinator.sh/bach-memory: 5Gi
```

负载均衡调度 - Load-Aware Scheduling

- **资源利用率均衡度影响运行时质量**
  - 负载热点和负载不均衡，影响工作负载的整体运行效果
  - 在线应用和离线任务之间冲突严重
- **可配置的调度插件控制集群利用率**
  - 自定义多维度资源和权重
  - 过滤负载过高的节点，选择利用率更低的节点
  - 基于资源预估和窗口机制，应对并发调度和冷节点场景





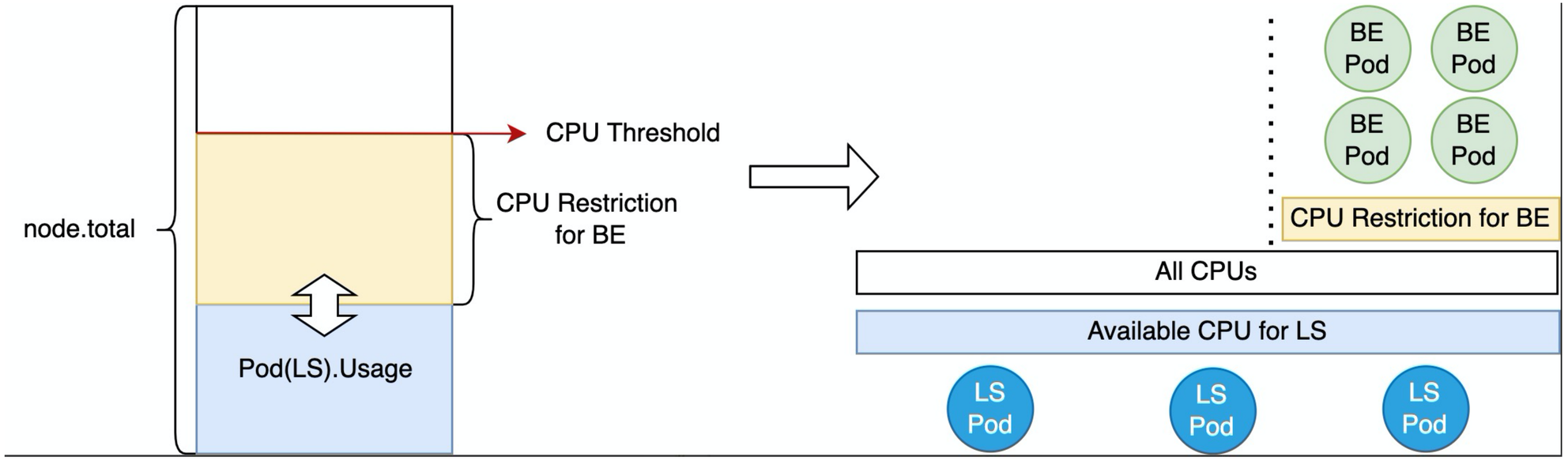
应用接入管理 – ClusterColocationProfile

- **ClusterColocationProfile CRD**
  - 降低 Koordinator 混部模型的使用门槛
  - 不侵入存量系统或组件
  - 按需针对 Namespace 和应用启用混部能力
  - 提供一键接入的能力，自动注入相关优先级、QoS 配置
- **举个例子**
  - 需要混部 Spark 作业
  - $ kubectl apply -f profile.yaml
  - $ kubectl label ns spark-job \
    -l coordinator.sh/enable-colocation=true
  - $ submit Spark Job
  - 所有带 Spark Operator 标签的 Pod 都会被注入相关混部协议

```yaml
apiVersion: config.koordinator.sh/v1alpha1
kind: ClusterColocationProfile
metadata:
  name: colocation-profile-example
spec:
  namespaceSelector:
    matchLabels:
      koordinator.sh/enable-colocation: "true"
  selector:
    matchLabels:
      sparkoperator.k8s.io/launched-by-spark-operator: "true"
  qosClass: BE
  priorityClassName: koord-batch
  koordinatorPriority: 1000
  schedulerName: koord-scheduler
  labels:
    koordinator.sh/mutated: "true"
  annotations:
    koordinator.sh/intercepted: "true"
  patch:
    spec:
      terminationGracePeriodSeconds: 30
```

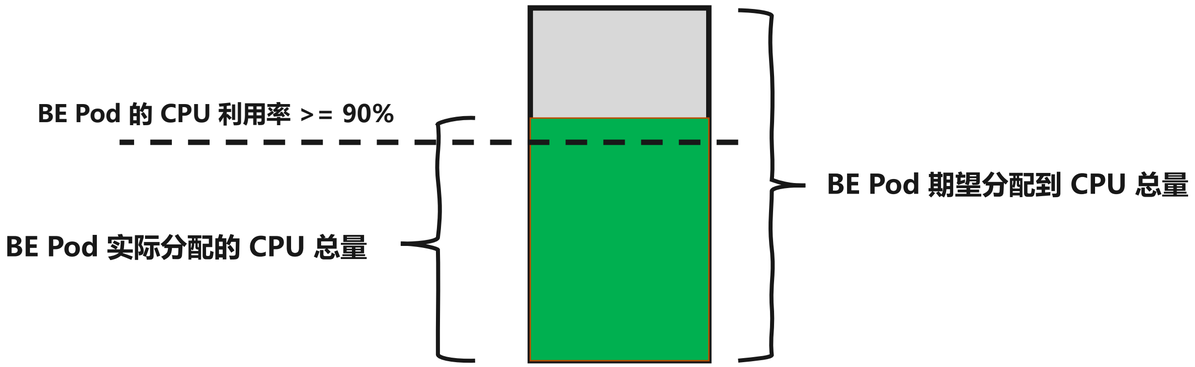
单机 QoS 增强 – CPU Suppress

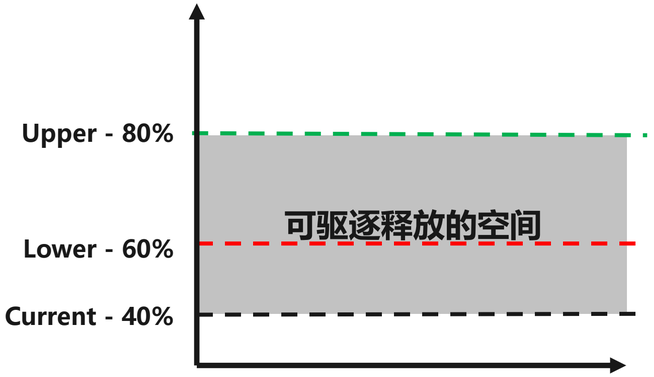
- BE Pod 的可用 CPU 取决于 LS Pod 的负载
- LS Pod 的负载升高时 ⬆，BE Pod 的可用 CPU 总量下降 ⬇，反之则增加



单机 QoS 增强 – 基于资源满足度的驱逐

- **CPU Suppress 有可能会频繁的压制 BE Pod**
  - BE Pod 的性能得不到满足
  - BE Pod 在被压制时持有内核全局锁等特殊资源，频繁压制可能导致优先级反转，影响 LS Pod 的性能
- **CPU 满足度**：实际分配的 CPU 总量 与 期望分配的 CPU 总量的比值

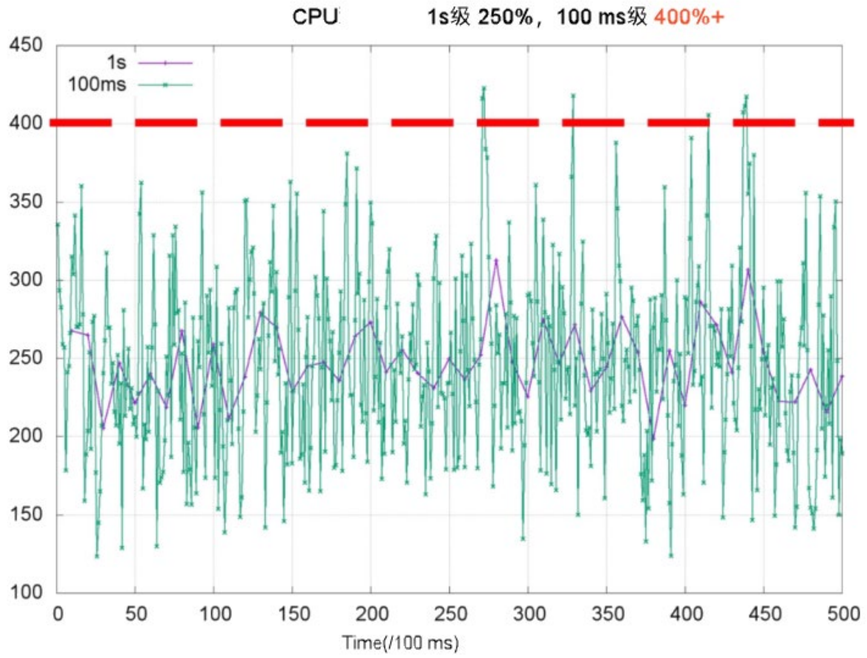


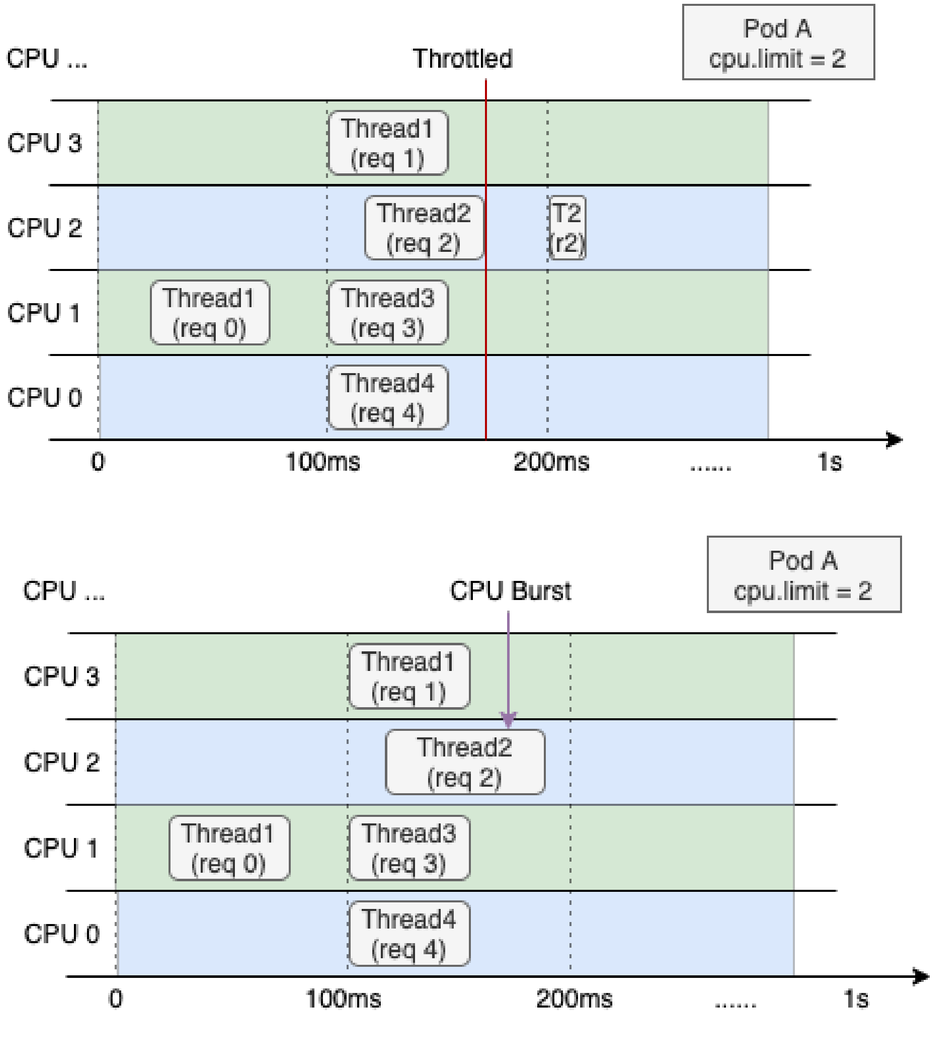


- **当 BE Pod 组的 CPU 得不到满足且 CPU 利用率超过阈值时驱逐**
  - 把节点上所有 BE Pod 作为一组统一决策
  - 同时参考 BE Pod 组最近一段时间的 CPU 利用率（Avg/Current）
  - 优先级越低的优先被驱逐，同优先级的利用率越高的优先被驱逐
  - 驱逐后会进入冷却期，防止过度驱逐

单机 QoS 增强 – CPU Burst

- **细粒度观察 CPU 利用率并不稳定**
  - CPU 突发和压制是常态
- **CPU Burst**
  - 空余的 CPU 资源将会被积累
  - 需要时使用这部分被积累的 CPU 资源





单机 QoS 增强 – Group Identity

- **Group Identity 特性是一种以 cgroup 组为单位实现调度特殊优先级的手段**
  - LS Pod 的任务唤醒延迟最小化
  - BE Pod 的任务唤醒不会对 LS Pod 造成性能影响
  - BE Pod 的任务不会通过 SMT 调度器共享物理核而对 LS Pod 造成性能影响

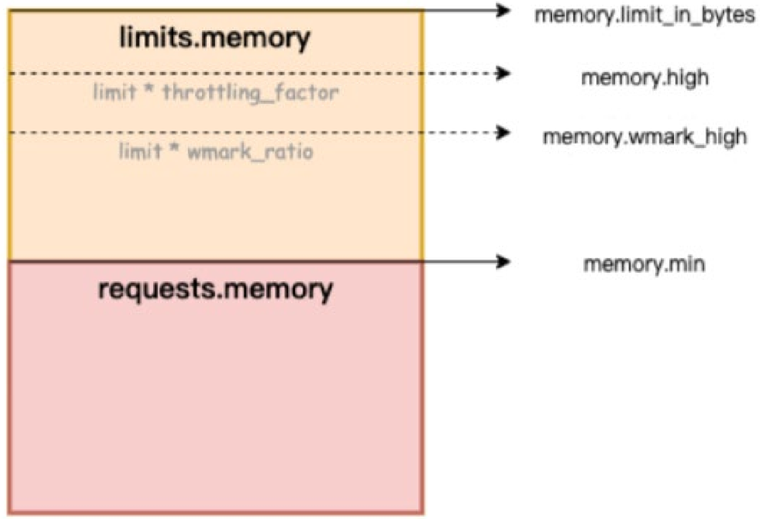
- **Group Identity 通过 cpu cgroup 接口 cpu.bvt_warp_ns 配置**

QoS	值	含义	说明
-	0	该 cgroup 中任务为普通系统任务	默认优先级
BE	-1	该 cgroup 中的任务为离线任务	优先级最低
-	1	该 cgroup 中的任务为在线任务	优先级高
LSR/LS	2	该 cgroup 中的任务为在线任务，并具有 SMT 驱逐能力	优先级最高

Alibaba Cloud Linux 2 从内核版本 kernel-4.19.91-24.al7 开始支持
More details, please refer to https://www.alibabacloud.com/help/zh/elastic-compute-service/latest/group-identity-feature

单机 QoS 增强 – Memory QoS

Interface	Description
minLimit	Absolutely locks the memory, which won’ t be reclaimed by kernel.
lowLimit	Relatively locks the memory, might be partially reclaimed if the system has no other memory to reclaim.
highThrottling	Limits the memory usage, memory allocation will be throllted if overused.
wmarkRatio	Reclaim memory asynchronously so that pod always have enough memory for new allocations.
wmarkMinAdj	Select BE pods preferentially during global memory reclaim by kernel, which will protect LS pods QoS.

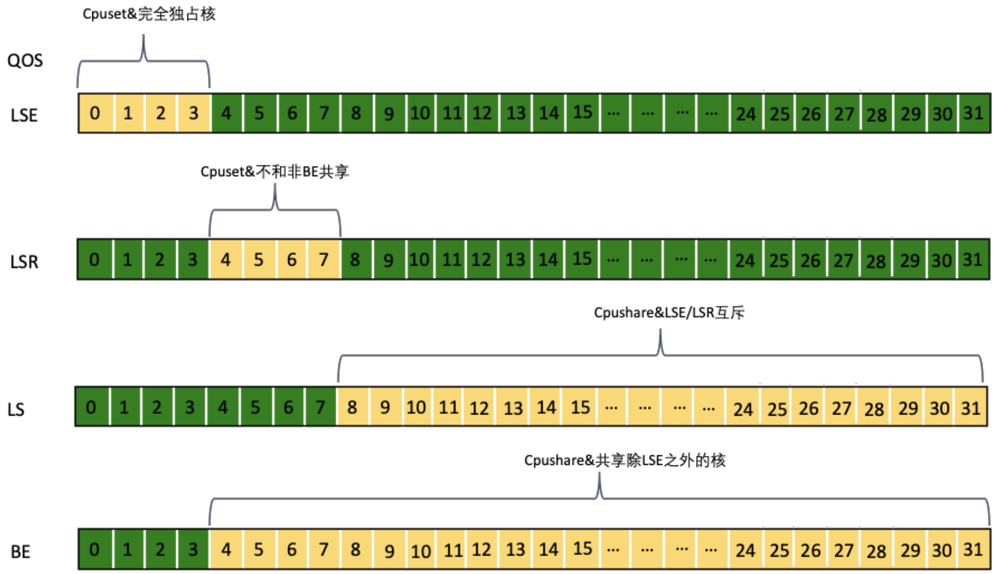


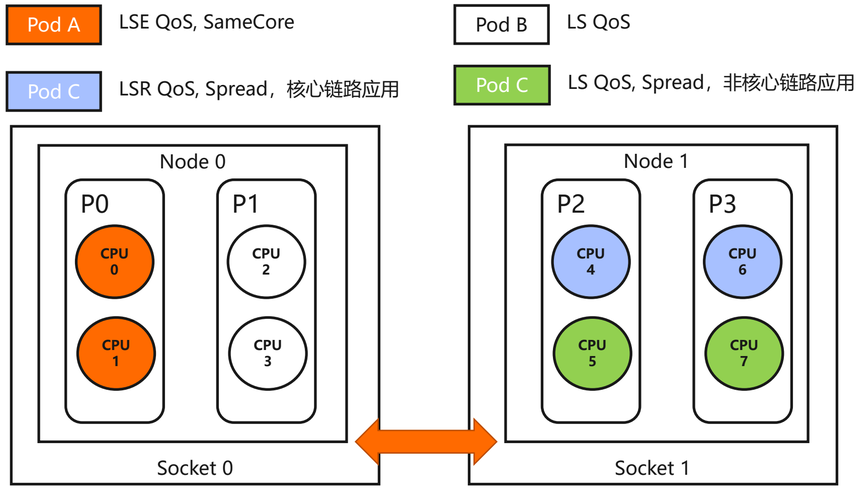
05 后续演进计划

精细化CPU编排 - Find-grained CPU Orchestration

- 更精确的 QoS 定义和丰富灵活的 CPU 编排策略
- 兼容 K8s QoS 和 已有的 CPU Manager 和 NUMA Topology Manager

QoS	K8s QoS	绑核策略	含义	说明
LSE	-	SameCore	完全独占 CPU，不与其他或相同QoS共享	满足少数对延迟要求苛刻的应用
LSR	Guaranteed	SameCore/ Spread	独占CPU，但可以被 BE 共享	满足大部分延时敏感型应用。当 kubelet CPU manager policy 为 static 时，Guaranteed 与 LSR 等价
LS	Guaranteed /Burstable	Automatically Resize	独立的CPU Shared Pool	满足大部分在线应用。当 kubelet CPU manager policy 为 none 时，Guaranteed 与 LS 等价
BE	BestEffort	BindAll	共享LSE独占之外的所有CPU	满足大部分离线型工作负载





- SameCore 策略
  - 更好的隔离性，但弹性空间小
- Spread 策略
  - 中等的隔离性，但可以通过其他隔离策略优化
  - 使用得当可以获得比 SameCore 策略更好的性能
  - 有一定的弹性空间

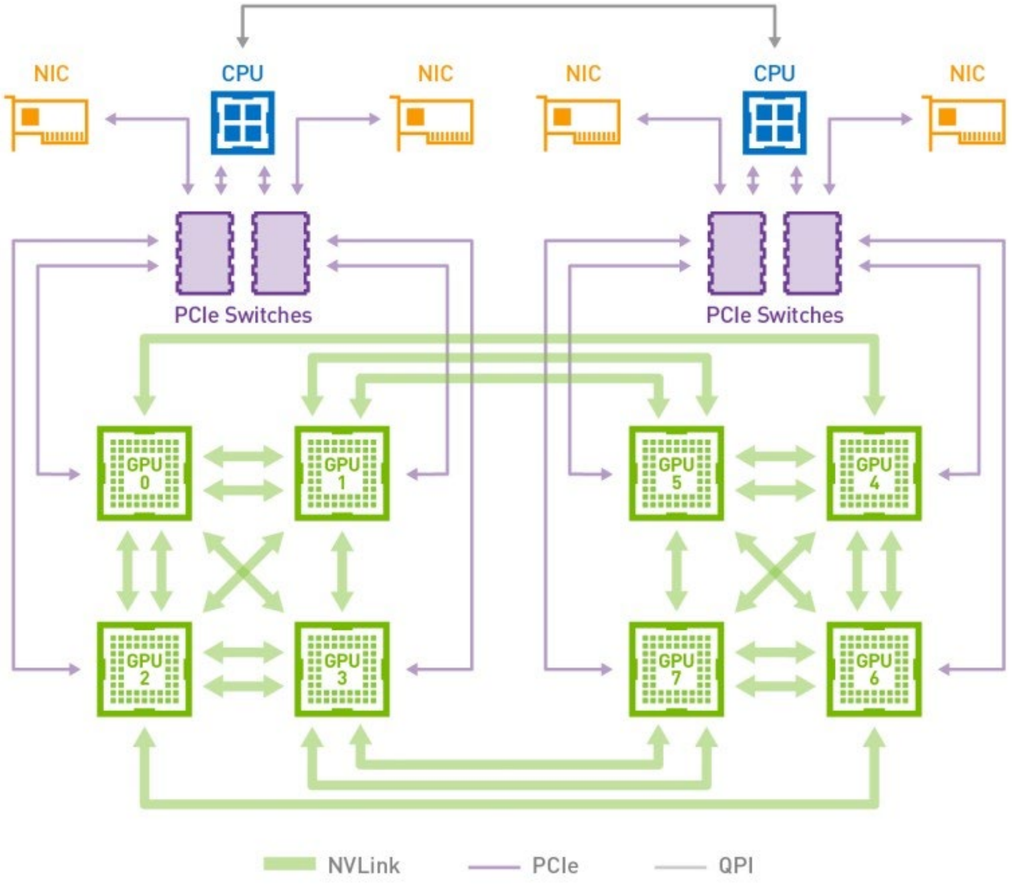
资源预留 - Resource Reservation

- **解决资源管理的痛点**
  - 提前预留资源应对峰值
  - 扩容前尝试预留资源，优化扩容体验
  - 优化抢占，确保抢占者可以拿到资源
  - 重调度时先预留资源再发起驱逐
- **特性**
  - 不侵入 K8s 社区已有的 API 和代码
  - 支持 PodTemplateSpec，在最合适的节点预留资源
  - 通过声明资源所有权连接 Pod 与 Reservation

```yaml
kind: Reservation
metadata:
  name: my-reservation
  namespace: default
spec:
  template: ... # a copy of the Pod's spec
  resourceOwners:
    controller:
      apiVersion: apps/v1
      kind: Deployment
      name: deployment-5b8df84dd
  timeToLiveInSeconds: 300 # 300 seconds
  nodeName: node-1
status:
  phase: Available
  ...
```

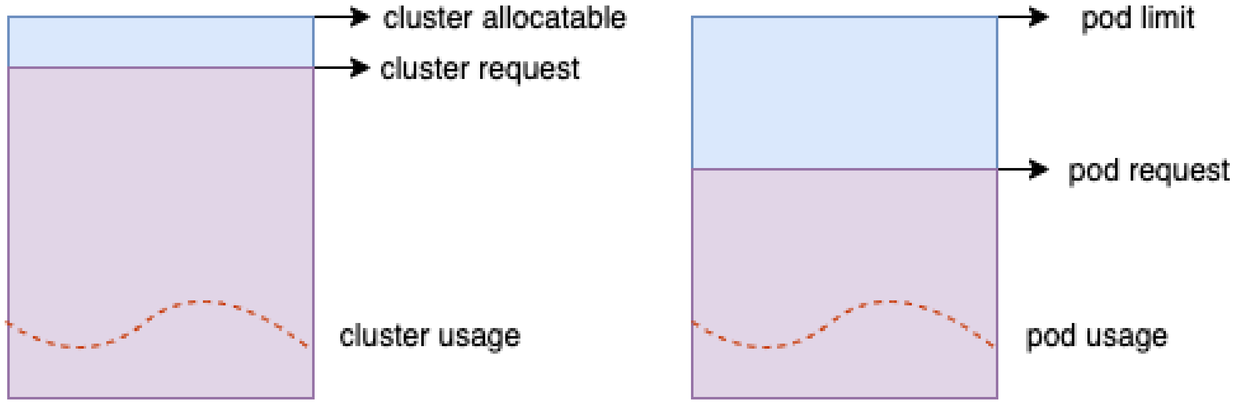
精细化 GPU 调度 - GPU Scheduling

- **GPU 拓扑感知调度**
  - 不同的拓扑组合获得不同的性能
  - 不同节点间的拓扑维度
  - 单节点内 GPU 卡之间的拓扑维度
- **GPU 扎堆调度**
  - 防止 GPU 卡维度碎片化
  - 同一 Job 的 worker Pod 尽量分配相同的卡
  - 优先用尽一台机器上的卡
- **CPU/GPU 混部优化**
  - CPU 计算类的任务尽量调度到纯 CPU 的节点
  - CPU 计算类的任务不要在 GPU 机器上扎堆
- ...

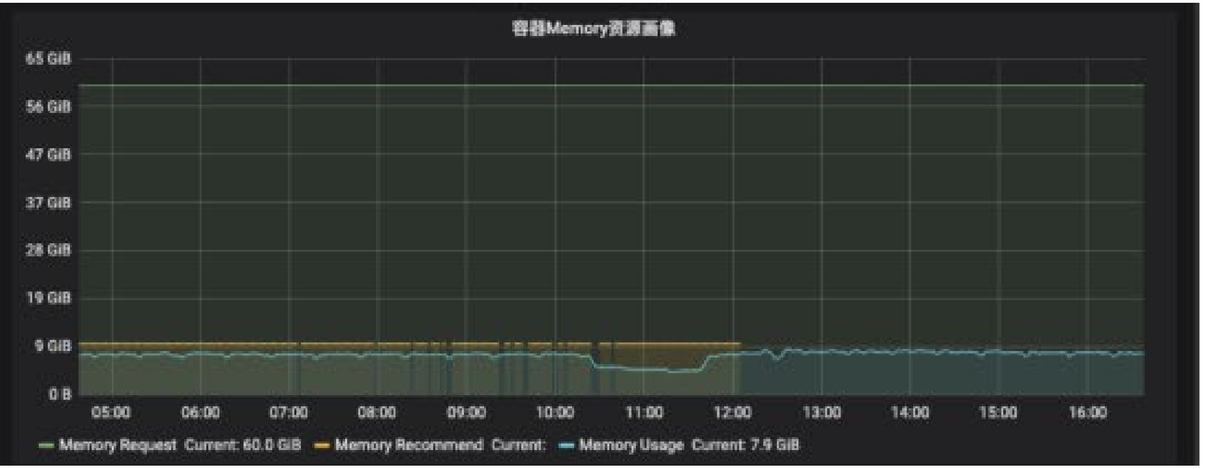


规格推荐 - Resource Recommendation

- **用户很难准确评估应用程序的资源使用情况**
  - 经常高估或者低估 Pod 资源规格
  - 导致资源浪费甚至稳定性风险
  - 工作负载多样，评估工作负荷大
- **Koordinator 资源画像**
  - 采集并加工分析工作负载历史指标数据
  - 推荐更准确的资源规格







06 社区建设

Koordinator Milestone

version	status	date	description
v0.1	Released	2022.03.30	• Resource overcommitment • Colocation Profile for application intergration • Dynamic CPU suppress policy
v0.2	Released	2022.04.19	• Isolate resources for best-effort workloads • Eviction mechanism based by memory safety thresholds • Module design vision of Koordlet
v0.3	Released	2022.05.07	• CPU Burst strategy • Design of Runtime Hooks
v0.4	Released	2022-05-31	• Runtime Proxy & Hooks • Load-aware scheduling • Group Identity • BE Pod evictions based on satisfaction • ...
v0.5	Ongoing		• Fine-grained CPU orchestration • Resource Reservation • ...
.....			• Descheduler • Gang Scheduling • GPU Scheduling • Elastic Quota • ...
			• Advanced scheduling polices for Co-location. • Integration with YARN, adapt with Hadoop.

Contributing to Koordinator

非常期待您在使用 Koordinator 积极的反馈遇到的任何问题、帮助改善文档、修复BUG和添加新功能

- If you find a typo, try to fix it!
- If you find a bug, try to fix it!
- If you find some redundant codes, try to remove them!
- If you find some test cases missing, try to add them!
- If you could enhance a feature, please DO NOT hesitate!
- If you find code implicit, try to add comments to make it clear!
- If you find code ugly, try to refactor that!
- If you can help to improve documents, it could not be better!
- If you find document incorrect, just do it and fix that!
- ...

官网：https://koordinator.sh/

双周会： 周二19:30~20:30
详情请关注社区交流群



微信群



钉钉群：33383887



© Copyright by Alibaba Cloud All rights reserved

WWW.ALIYUN.COM